

Network Programming

元培数科 王恩博

Let's get started →

2025/12/17



Knowledge Review

客户端 - 服务端模型

Client - Server Model

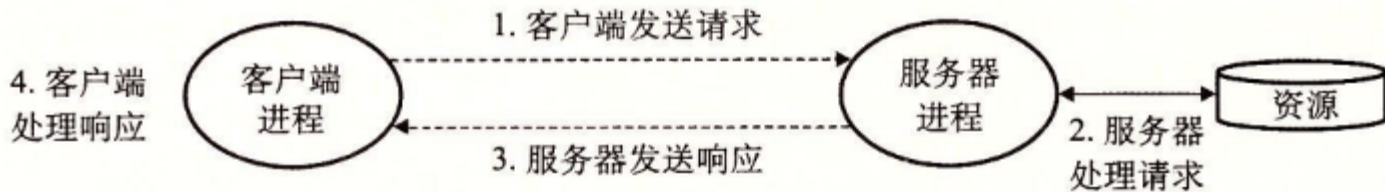


图 11-1 一个客户端-服务器事务

客户端和服务端是 **进程**，而不是 **主机**，它们可以在同一台主机上，也可以在不同的主机上。

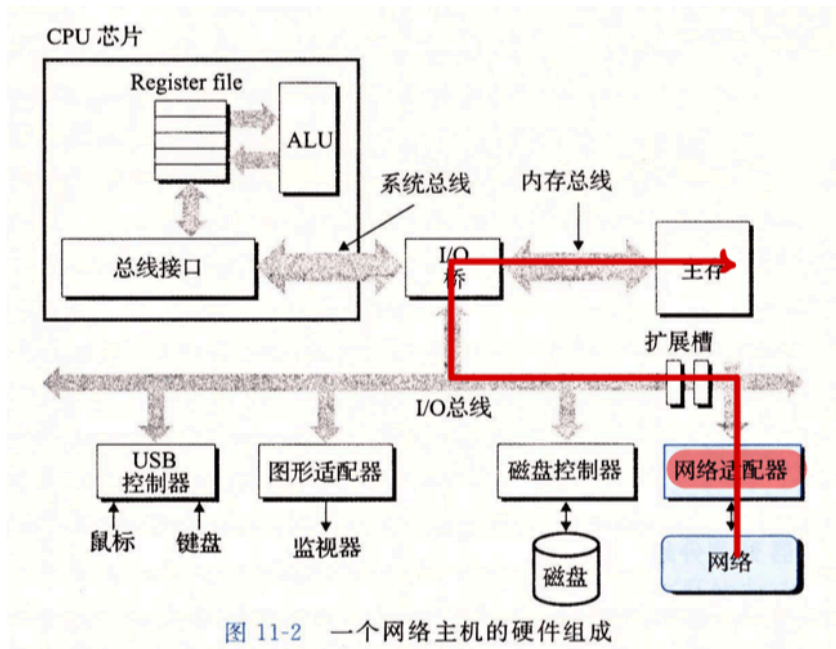
但是一般而言，你可以这么理解网络：到另一台计算机上获取文件（资源）

网络

Network

网络是一种 I/O 设备，通常使用 DMA 传输。

网络是按照地理远近组成的层次系统。



网络

Network

局域网 (Local Area Network, LAN)

以太网 (Ethernet) 是一种 LAN 技术

- 主机
- 集线器 (Hub)：不加分辨地转发到所有端口
- 桥 (Bridge)：存在分配算法，有选择性地转发

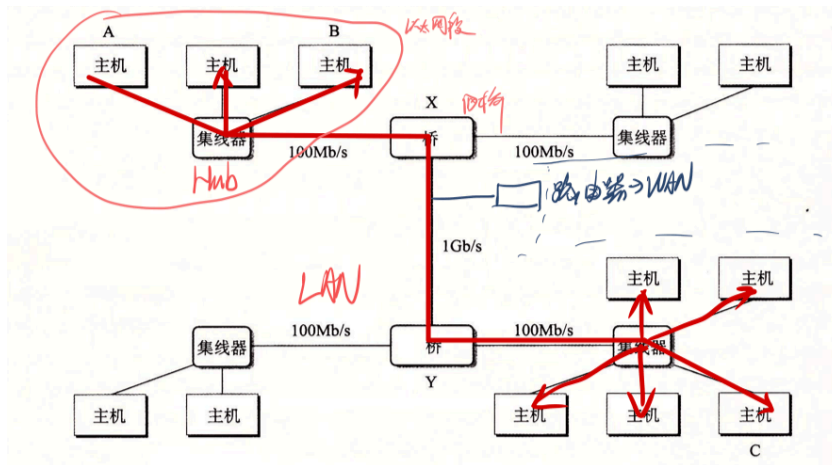
LAN 帧 = LAN 帧头 + 互联网络包 (有效载荷)

广域网 (Wide Area Network, WAN)

WAN 由路由器连接若干 LAN 组成。

- 路由器 (Router)：每台路由器对它所连接到的每个网络都有一个适配器 (端口)

互联网络包 = 互联网络包头 + 用户数据 (有效载荷)



桥接 LAN

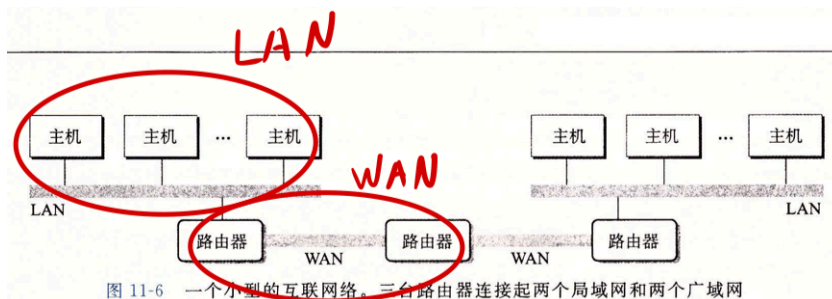


图 11-6 一个小型的互联网络。三台路由器连接起两个局域网和两个广域网

WAN

互联网

Internet

互联网 (internet) : 路由器连接若干 LAN 和 WAN

全球 IP 因特网 (Internet) : 是互联网 (internet) 的具体实现

协议软件

协议软件: 让主机能够跨过不兼容的网络发送数据

- 命名机制
- 传送机制: 运行在主机和路由器上, 控制主机和路由器协同工作。
一个包是由 **包头** 和 **有效载荷** 组成的, 包头包括了传输所需信息, 如源主机地址、目的主机地址

互联网络传输

internet transfer

你可能会疑惑：“网络”在哪里？

注意这是一个 internet 而不是 Internet，它只有两个 LAN。

- 帧的有效载荷是互联网络包
- 互联网络包的有效载荷是数据

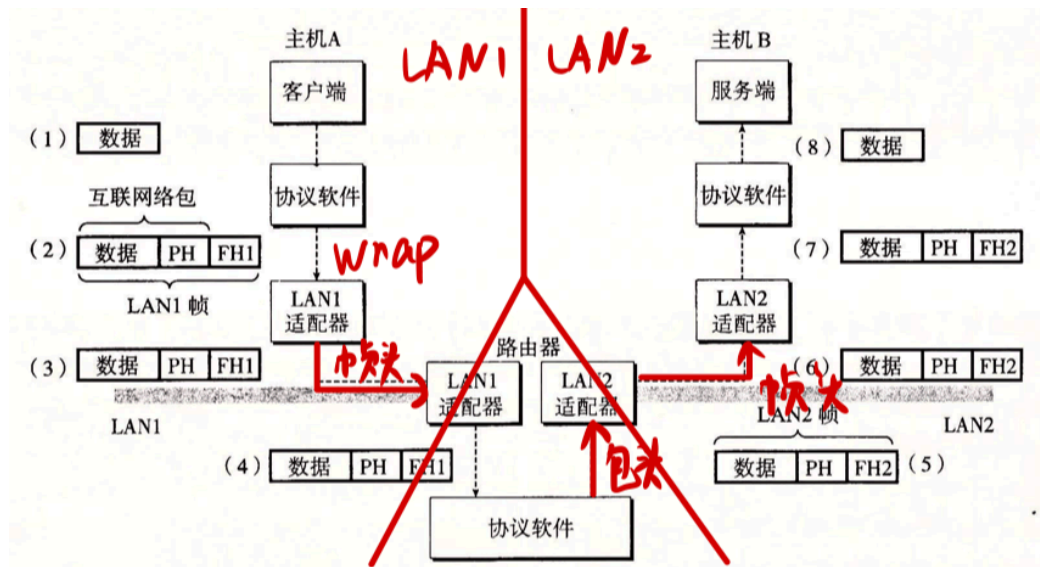


图 11-7 在互联网络上，数据是如何从一台主机传送到另一台主机的 (PH：互联网络包头；FH1：LAN1 的帧头；FH2：LAN2 的帧头)

全球 IP 因特网

Internet

TCP / IP 协议

- 内核模式运行
- 是协议族，提供不同的功能

Key points:

- 主机集合被映射为一组 32 位的 **IP 地址**
- 这组 IP 地址被映射为一组称为因特网 **域名** (Internet domain name) 的标识符
- 因特网主机上的进程能够通过连接 (connection) 和任何其他因特网主机上的进程通信。

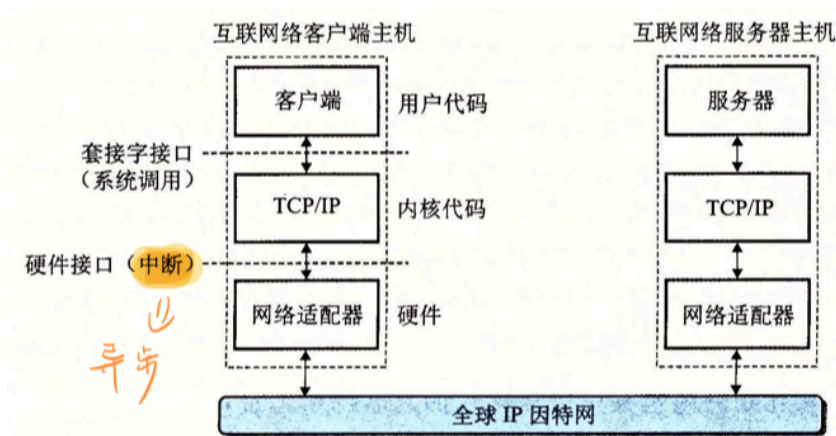


图 11-8 一个因特网应用程序的硬件和软件组织

全球 IP 因特网

Internet

IP协议 (Internet Protocol, 互联网协议) :

- 它提供了一种基本的命名方法和传递机制 (给每台电脑分配一个唯一的 32 位 IP 地址, 并负责把数据包 (或者叫数据宝, Datagram) 送到正确的地址)
- 它是不可靠的 (不保证数据包一定会到达目的地, 也不负责重传丢失的数据包)

UDP协议 (User Datagram Protocol, 用户数据报协议) 是对 IP 协议的扩展:

- 它允许数据包在不同的进程之间传送 (就像允许不同的程序之间直接发送数据)
- 它也不保证数据的可靠传输

TCP协议 (Transmission Control Protocol, 传输控制协议) 是构建在 IP 协议之上的更复杂的工具:

- 它提供了可靠的 **全双工连接** (就像在两个程序之间建立一条可靠的双向通道, 确保数据准确无误地传输)

全球 IP 因特网

Internet

网络字节顺序：**大端法**（可能需要相关函数进行转换）

IP 地址：点分十进制表示法（如 `128.2.194.242`） / 十六进制 IP 地址（`0x8002c2f2`）

Linux 命令 `hostname -i` 可以查看主机的点分十进制地址

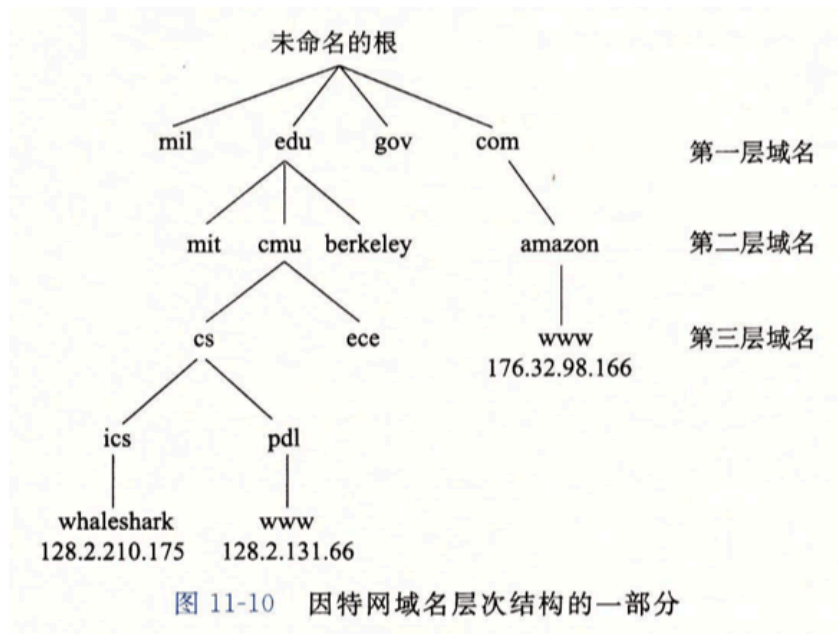
注意：**点分十进制表示法是以字符串形式存储的**

因特网域名

Internet domain name

域名是多层次的: ics.huh.moe huh.moe moe

DNS (Domain Name System, 域名系统) 维护域名集合和 IP 地址的映射



因特网域名

Internet domain name

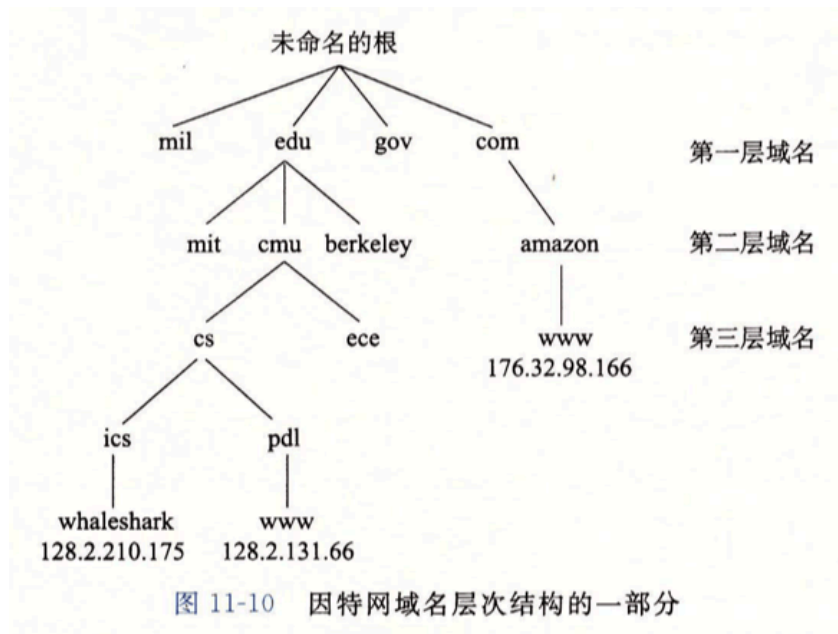
域名

- 一个域名可以解析到多个 IP 地址
- 一个 IP 地址可以对应多个域名
- 某些合法的域名没有映射到任何 IP 地址
- `localhost` 回送地址 `127.0.0.1`

端口

表示服务类型，由 `/etc/services` 文件维护

- 22: SSH 远程登录
- 80: HTTP Web 端口
- 443: HTTPS Web 端口



套接字

Socket

套接字 (Socket)：提供了一层抽象，让网络 I/O 对于进程而言就像文件 I/O 一样，可以类比文件描述符。

然而，套接字还是与传统的文件 I/O 有所差异，在使用前需要调用一些特殊函数

套接字是连接的一个 **端点**，一对套接字组成一个连接。

套接字地址形如 (IP 地址:端口号)

套接字对形如 (客户端 IP:客户端端口, 服务端 IP:服务端端口)，其唯一确定了一个连接。

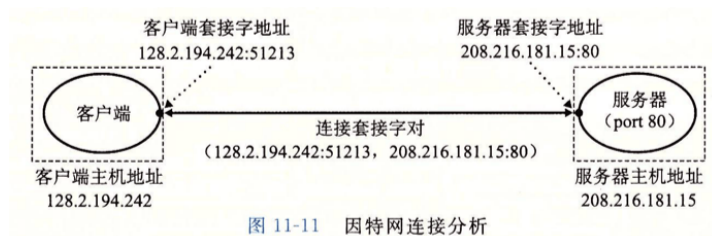


图 11-11 因特网连接分析

套接字接口

Socket interface

类型定义

套接字接口是一组函数，和 Unix I/O 函数结合创建网络应用。

`sin` 前缀：Socket Internet

IP 地址结构中存放的地址总是以（大端法）网络字节顺序存放的

```
/* IP 地址结构，2B */
struct in_addr {
    uint32_t s_addr; /* 网络字节顺序的地址（大端序） */
};

/* 套接字地址结构，16B */
typedef struct sockaddr SA;
struct sockaddr {
    uint16_t sa_family; /* 协议族，2B，如 AF_INET（IPv4）、AF_INET6（IPv6） */
    char sa_data[14]; /* 存储具体的地址数据，14B，与协议族相关 */
}; // 整体 16B

/* 套接字地址结构，16B */
struct sockaddr_in {
    uint16_t sin_family; /* 协议族（总是 AF_INET） */
    uint16_t sin_port; /* 端口号（网络字节序） */
    struct in_addr sin_addr; /* IP 地址（网络字节序） */
    unsigned char sin_zero[8]; /* 填充到 sizeof(struct sockaddr) = 16B */
}; // 兼容 sockaddr，整体 16B
```

套接字接口

Socket interface

客户端

- `getaddrinfo` : 获取地址信息
- `socket` : 创建套接字
- `connect` : 连接到服务器
- `rio_writen` : 发送 n 字节数据
- `rio_readlineb` : 读取一行数据
- `close` : 关闭套接字

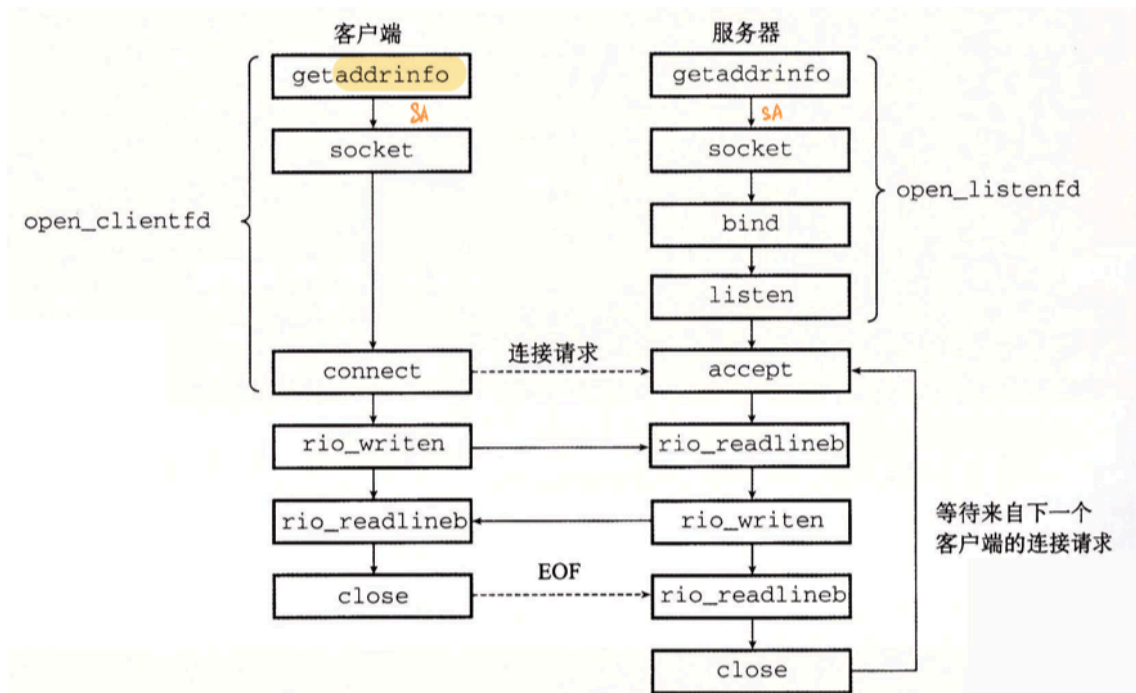


图 11-12 基于套接字接口的网络应用概述

套接字接口

Socket interface

服务器

- `getaddrinfo` : 获取地址信息
- `socket` : 创建套接字
- `bind` : 绑定套接字到地址
- `listen` : 监听套接字
- `accept` : 接受连接
- `rio_readlineb` : 读取一行数据
- `rio_writen` : 发送 n 字节数据
- `close` : 关闭套接字

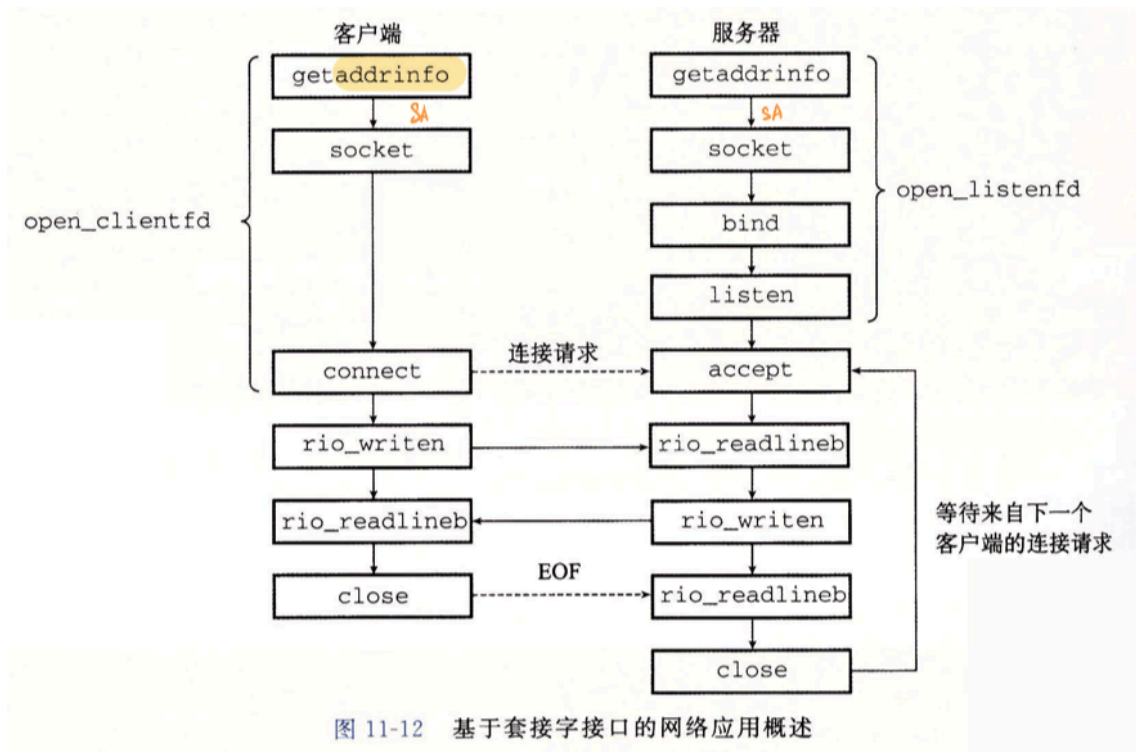


图 11-12 基于套接字接口的网络应用概述

套接字接口

Socket interface

为什么要搞得这么复杂？

网络编程不像文件 I/O 那么“确定”

对于客户端：

1. 为了得到连接对方的信息，我们需要先用 `getaddrinfo` 获取对方的网络连接信息。
2. 然后我们需要用 `socket` 创建一个套接字，但是这只是填入了对方的基础信息，我们不确定对方的状态，所以此时套接字是 **不可用** 的。
3. 我们需要调用 `connect` 来尝试建立连接，若能正常建立，则得知此时套接字是 **可用** 的。
4. 然后我们就可以调用 `rio_writen` 和 `rio_readlineb` 来发送和接收数据了。
5. 最后我们调用 `close` 关闭套接字，终止连接，释放描述符使之可以被重用。

套接字接口

Socket interface

对于服务端：

1. 为了能够参与到网络连接，我们需要先用 `getaddrinfo` 获取自己的网络连接信息。
2. 然后我们需要用 `socket` 创建一个套接字，用于监听客户端的连接请求，此时，这个套接字依旧是 **不可用** 的。
3. 调用 `bind` 将套接字绑定到获取到的地址信息上，声明对某个端口（如 HTTP 对 80 端口）的占用，这样套接字就可以使用了。
4. 调用 `listen` 来将之转为监听套接字，这样套接字就可以监听客户端的连接请求了。
5. 接下来，我们调用 `accept` 来从监听序列中取出一个套接字，这代表我们接受了客户端的连接请求，这样，我们就可以使用这个套接字和客户端进行通信了。
6. 调用 `rio_readlineb` 和 `rio_writen` 来发送和接收数据。
7. 当读到客户端调用 `close` 关闭连接的信息后，我们也调用 `close` 关闭套接字，从而允许这个套接字被重用。

你可能会疑惑，为什么不能合并 `bind` 和 `listen`？它们似乎都只在服务端使用。

- `bind` 代表绑定，它使得侦听端口固定了下来。在客户端不需要这个过程，每次连接随便分配一个可用端口即可。
- `listen` 代表监听，它让套接字从主动套接字变为被动套接字。

套接字接口

Socket interface

getaddrinfo 函数用于获取地址信息

```
int getaddrinfo(const char *host, const char *service, const struct addrinfo *hints, struct addrinfo **res);
void freeaddrinfo(struct addrinfo *res);
```

- **host** : 主机名, 可以是 127.0.0.1 或 localhost , 域名或点分制都可以, 甚至可以是 NULL , 代表监听本机通配地址 0.0.0.0 (监听所有网络接口)
- **service** : 服务名, 可以是 80 (十进制端口号) 或 http , 端口号或服务名都可以
- **hints** : 提示信息, 控制行为
- **res** : 地址信息链表, 存放结果

ai 是 addrinfo 的缩写

canonicalname 全称 canonical name, 即规范、权威名 返回的字段可以直接传给后面的套接字接口函数。

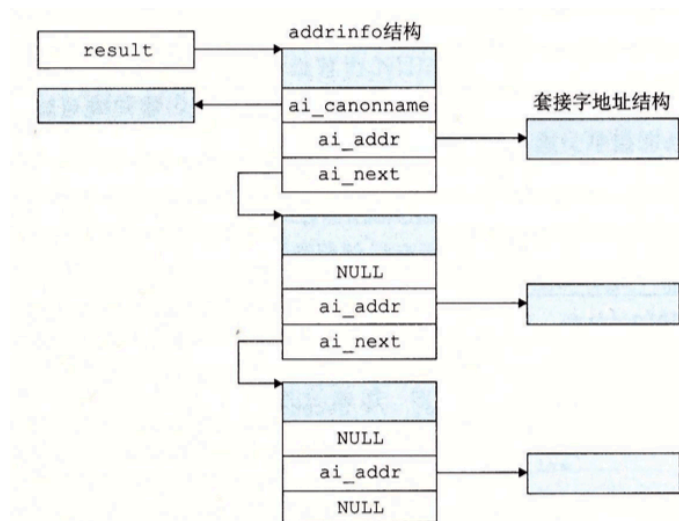


图 11-15 getaddrinfo 返回的数据结构

套接字接口

Socket interface

hints 参数

- `ai_family` 设置为 `AF_INET` 将列表限制为IPv4地址，设置为 `AF_INET6` 将列表限制为IPv6地址。
- `ai_socktype` 设置为 `SOCK_STREAM` 将列表限制为每个地址最多一个 `addrinfo` 结构。
- `ai_flag` 可以用or组合的掩码：
- `AI_ADDRCONFIG` 使用连接时建议使用。要求只有当本地主机配置为IPv4时，`getaddrinfo` 返回IPv4地址。IPv6同理
- `AI_CANONNAME` 把列表中第一个 `addrinfo` 结构的 `ai_canonname` 指向host的权威名字。（默认为NULL）
- `AI_NUMERICSERV` 强制参数 `service` 为端口号。
- `AI_PASSIVE` 告诉函数返回的套接字地址可能被服务器用作监听套接字，此时host应为NULL。得到的地址字段是通配符地址，会接受发到该主机所有IP地址的请求。

套接字接口

Socket interface

getnameinfo 函数用于获取主机和服务名字符串

```
int getnameinfo(const struct sockaddr *sa, socklen_t salen, char *host, size_t hostlen,
char *service, size_t servlen, int flags);
```

- host 指向大小为 hostlen 字节的缓冲区， service 指向大小为 servlen 字节的缓冲区。
- 把套接字地址结构 sa 转换为对应的主机和服务名字符串，并复制到相应的缓冲区。
- 可以把 host 或 service 设置为NULL，并把相应的长度设置为0。（至少一个不为NULL）
- flags 为掩码：
- NI_NUMERICHOST 返回数字地址字符串。（默认为域名）
- NI_NUMERICSERV 返回端口号。（默认为检查 /etc/services，尽可能返回服务名）

套接字接口

Socket interface

`socket` 函数用于创建可供读写的套接字。

```
int socket(int domain, int type, int protocol);
```

- `domain` : 协议族, 如 `AF_INET` / `AF_INET6`
- `type` : 套接字类型, 如 `SOCK_STREAM` / `SOCK_DGRAM`
- `protocol` : 协议, 如 `0`

使用 `getaddrinfo` 获取地址信息后, 可以自动生成这些所需参数。

套接字接口

Socket interface

`connect` 函数用于建立和服务器的连接。

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- `sockfd` : 套接字描述符
- `addr` : 套接字地址
- `addrlen` : 套接字地址长度

注意, `connect` 函数会阻塞, 直到连接建立成功或失败。

而且还要注意, `connect` 是否返回, 与服务器是否 `accept` 无关。

前者返回 `0` 表明建立了连接, 后者代表开始服务器处理这个连接。

套接字接口

Socket interface

`bind` 函数用于在服务器将套接字绑定到地址。它声明对某个端口（如 HTTP 对 80 端口）的占用。

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- `sockfd` : 套接字描述符
- `addr` : 套接字地址
- `addrlen` : 套接字地址长度

依旧是最好使用 `getaddrinfo` 获取地址信息后，可以自动生成这些所需参数。

套接字接口

Socket interface

`listen` 函数用于在服务器将套接字转为监听套接字。该端口必须首先被声明占用（`bind`）。

```
int listen(int sockfd, int backlog);
```

- `sockfd`：套接字描述符
- `backlog`：连接队列长度，**表示服务器最多可以同时处理多少个连接。**

调用完 `listen` 后，对于连接请求，会放入队列。

回忆：这是一个队列，建立连接后，我们先放进去，但不代表我们立即调用 `accept` 来处理。

监听描述符只被创建一次，存在于整个生命周期。

套接字接口

Socket interface

`accept` 函数用于在服务器接受连接。

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

- `sockfd` : 监听套接字描述符
- `addr` : 客户端套接字地址会被填写在这里
- `addrlen` : 客户端套接字地址长度

从队列取出连接请求，如果决定接受，则为该连接分配新的描述符 `fd`

- 多个连接存在时，每个连接有各自的 `fd`，但共用一个端口号，即监听时的端口号。
- 系统内部用（源 IP 地址，源端口号，目的 IP 地址，目的端口号）区分连接
- 因此，即使多个连接共用服务器端口号，也能区分
- 即使原先监听描述符被提前关闭，也不影响已经建立的连接。

已连接描述符只存在于为一个客户端服务的过程中。

监听描述符 vs 已连接描述符

listen fd vs connected fd

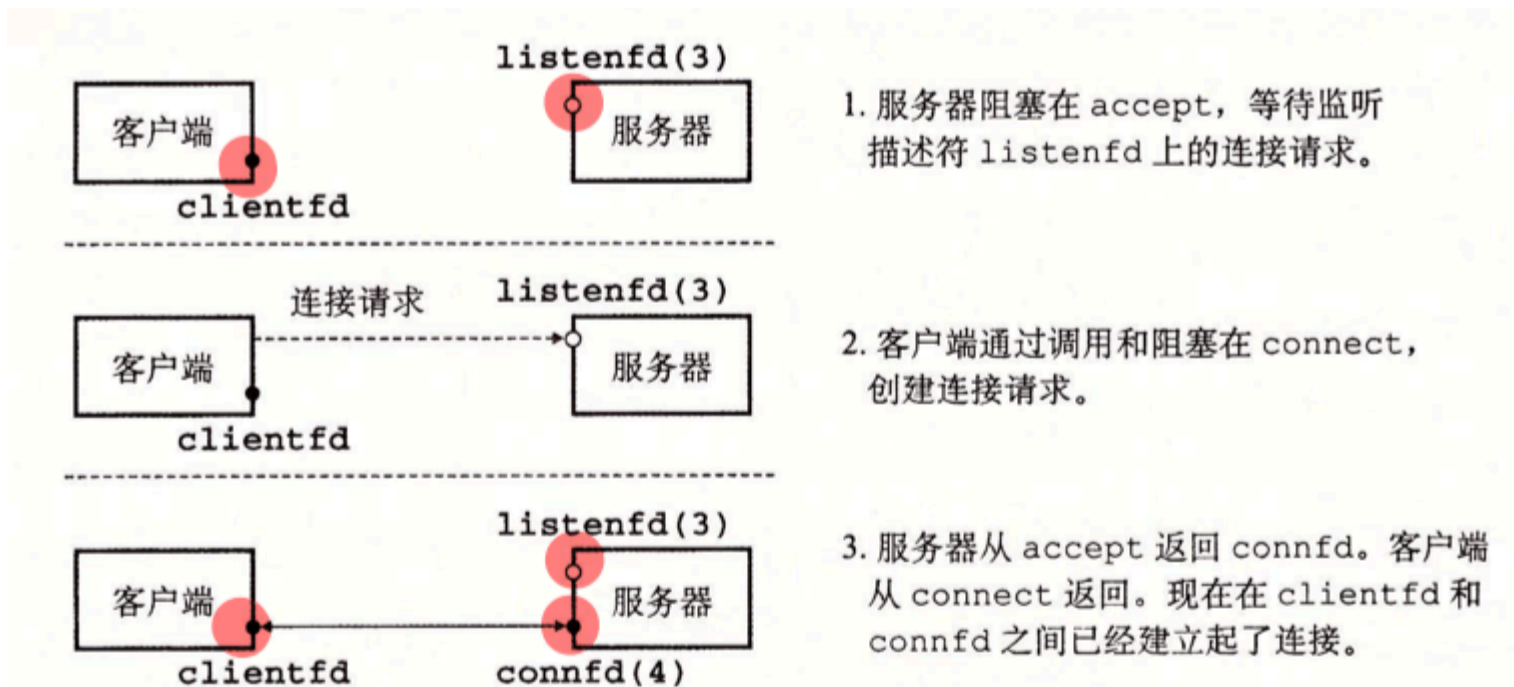


图 11-14 监听描述符和已连接描述符的角色

套接字接口辅助函数

Socket interface helper functions

`open_clientfd`

```
int open_clientfd(char *hostname, char *port);
```

客户端调用 `open_clientfd` 函数建立与服务器的连接，该服务器运行在主机 `hostname` 上，在端口号 `port` 上监听连接请求。

等价于 `getaddrinfo` + `socket` + `connect`

`open_listenfd`

```
int open_listenfd(char *port);
```

服务端调用 `open_listenfd` 函数，服务器创建一个监听描述符，准备好接收连接请求。

等价于 `getaddrinfo` + `socket` + `bind` + `listen`

```

int open_clientfd(char *hostname, char *port) {
    int clientfd, rc;
    struct addrinfo hints, *listp, *p;

    /* Get a list of potential server addresses */
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_socktype = SOCK_STREAM; /* Open a connection */
    hints.ai_flags = AI_NUMERICSERV; /* ... using a numeric port arg. */
    hints.ai_flags |= AI_ADDRCONFIG; /* Recommended for connections */
    if ((rc = getaddrinfo(hostname, port, &hints, &listp)) != 0) {
        fprintf(stderr, "getaddrinfo failed (%s:%s): %s\n", hostname, port, gai_strerror(rc));
        return -2;
    }

    /* Walk the list for one that we can successfully connect to */
    for (p = listp; p; p = p->ai_next) {
        /* Create a socket descriptor */
        if ((clientfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
            continue; /* Socket failed, try the next */

        /* Connect to the server */
        if (connect(clientfd, p->ai_addr, p->ai_addrlen) != -1)
            break; /* Success */

        if (close(clientfd) < 0) { /* Connect failed, try another */ //line:netp:openclientfd:closefd
            fprintf(stderr, "open_clientfd: close failed: %s\n", strerror(errno));
            return -1;
        }
    }
}

```

```

int open_listenfd(char *port)
{
    struct addrinfo hints, *listp, *p;
    int listenfd, rc, optval=1;

    /* Get a list of potential server addresses */
    memset(&hints, 0, sizeof(struct addrinfo));
    hints.ai_socktype = SOCK_STREAM;          /* Accept connections */
    hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG; /* ... on any IP address */
    hints.ai_flags |= AI_NUMERICSERV;         /* ... using port number */
    if ((rc = getaddrinfo(NULL, port, &hints, &listp)) != 0) {
        fprintf(stderr, "getaddrinfo failed (port %s): %s\n", port, gai_strerror(rc));
        return -2;
    }

    /* Walk the list for one that we can bind to */
    for (p = listp; p; p = p->ai_next) {
        /* Create a socket descriptor */
        if ((listenfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol)) < 0)
            continue; /* Socket failed, try the next */

        /* Eliminates "Address already in use" error from bind */
        setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, //line:netp:csapp:setsockopt
                   (const void *)&optval, sizeof(int));

        /* Bind the descriptor to the address */
        if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)

```

```
/* Clean up */  
freeaddrinfo(listp);  
if (!p) /* No address worked */  
    return -1;  
  
/* Make it a listening socket ready to accept connection requests */  
if (listen(listenfd, LISTENQ) < 0) {  
    close(listenfd);  
    return -1;  
}  
return listenfd;  
}
```

echo 客户端

```
int main(int argc, char **argv)
{
    int clientfd;
    char *host, *port, buf[MAXLINE];
    rio_t rio;

    if (argc != 3) {
        fprintf(stderr, "usage: %s <host> <port>\n", argv[0]);
        exit(0);
    }
    host = argv[1];
    port = argv[2];

    clientfd = Open_clientfd(host, port);
    Rio_readinitb(&rio, clientfd);

    while (Fgets(buf, MAXLINE, stdin) != NULL) {
        Rio_writen(clientfd, buf, strlen(buf));
        Rio_readlineb(&rio, buf, MAXLINE);
        Fputs(buf, stdout);
    }
    Close(clientfd); //line:netp:echoclient:close
    exit(0);
}
```

echo 服务器

```
int main(int argc, char **argv)
{
    int listenfd, connfd;
    socklen_t clientlen;
    struct sockaddr_storage clientaddr; /* Enough space for any address */
    char client_hostname[MAXLINE], client_port[MAXLINE];

    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(0);
    }

    listenfd = Open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(struct sockaddr_storage);
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        Getnameinfo((SA *)&clientaddr, clientlen, client_hostname, MAXLINE,
                    client_port, MAXLINE, 0);
        printf("Connected to (%s, %s)\n", client_hostname, client_port);
        echo(connfd);
        Close(connfd);
    }
    exit(0);
}
```

echo 程序

```
#include "csapp.h"

void echo(int connfd)
{
    size_t n;
    char buf[MAXLINE];
    rio_t rio;

    Rio_readinitb(&rio, connfd);
    while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) { //line:netp:echo:eof
        printf("server received %d bytes\n", (int)n);
        Rio_writen(connfd, buf, n);
    }
}
```

- 注意网络中使用rio包
- 对于网络，当一个进程关闭连接它的那一段时，会发生EOF
- 连接另一端的进程在试图读取流中最后一个字节之后的字节时会检测到EOF

Web

- 超文本传输协议：HTTP
- Web内容用HTML（超文本标记语言）语言编写
- 内容是和一个MIME（多用途的网络邮件扩充协议）类型相关的字节序列

MIME type	Description
text/html	HTML page
text/plain	Unformatted text
application/postscript	Postscript document
image/gif	Binary image encoded in GIF format
image/png	Binary image encoded in PNG format
image/jpeg	Binary image encoded in JPEG format

Web

- 服务静态内容：服务器给客户端提供静态内容（磁盘文件）
- 服务动态内容：服务器给客户端提供动态内容（执行程序返回输出）
- URL（通用资源定位符）：[https://www.chinesemooc.org/student/task_exam.php?courseid=4392&termid=2992&name=计算机组成 - 2025秋季校内课学期](https://www.chinesemooc.org/student/task_exam.php?courseid=4392&termid=2992&name=计算机组成-2025秋季校内课学期)
- <http://120.232.255.236:30637/login?next=%2F>
- ? 分割文件名和参数， & 分隔多个参数
- 前缀客户端用，后缀服务器用
- / 代表主目录，为最小后缀，服务器自行扩展为某个主页，浏览器可以自动补 /

HTTP 事务

HTTP Transactions

http响应：请求行+请求报头+空行+响应主体

相应行：version + status-code + status-message

请求报头：HOST报头在 HTTP/1.1请求中是需要的，在HTTP/1.0不需要。用于指示原服务器域名，让代理判断是否已经在本地拥有副本。

Status code	Status message	Description
200	OK	Request was handled without error.
301	Moved permanently	Content has moved to the hostname in the Location header.
400	Bad request	Request could not be understood by the server.
403	Forbidden	Server lacks permission to access the requested file.
404	Not found	Server could not find the requested file.
501	Not implemented	Server does not support the request method.
505	HTTP version not supported	Server does not support version in request.

▼ 响应标头

Accept-Ranges	bytes
Content-Length	3977
Content-Type	image/png
Date	Mon, 15 Dec 2025 17:02:12 GMT 
Etag	"68d3b647-f89"
Last-Modified	Wed, 24 Sep 2025 09:13:43 GMT
Strict-Transport-Security	max-age=31536000; includeSubDomains

```
HTTP/1.0 200 OK
MIME-Version: 1.0
Date: Mon, 8 Jan 2010 4:59:42 GMT
Server: Apache-Coyote/1.1
Content-Type: text/html
Content-Length: 42092
```

```
<html>
...
</html>
```

服务动态内容

Serving Dynamic Content

- 利用CGI (Common Gateway Interface, 通用网关接口)
- 传参方法: GET在URI中, 而POST在主体中

▼ 常规	
请求 URL	http://120.232.255.236:30637/login?next=%2F
请求方法	POST
状态代码	● 302 FOUND
远程地址	120.232.255.236:30637
引用站点策略	strict-origin-when-cross-origin

X 标头 负载 预览 响应 发起程序 计时 Cookie	
▼ 查询字符串参数 查看源 查看 URL 编码	
next	/
▼ 表单数据 查看源 查看 URL 编码	
salt	XfRGIFgmeEEL71al
password	d1657748cb88ad2c84d50b3c0b8aa65d4458d
username	wangenbo

- 服务器收到请求后, 创建子进程执行程序, 会把参数设置环境变量 `QUERY_STRING`
- 子进程运行前先把标准输出重定向到已连接描述符, 进而将标准输出送到客户端。
- CGI程序需要生成**部分响应报头**和空行。(换行为 `\r\n`)
- CGI中可以正常使用标准I/O函数

CGI

```
int main(void) {
    char *buf, *p;
    char arg1[MAXLINE], arg2[MAXLINE], content[MAXLINE];
    int n1=0, n2=0;
    if ((buf = getenv("QUERY_STRING")) != NULL) {
        p = strchr(buf, '&');
        *p = '\0';
        strcpy(arg1, buf);strcpy(arg2, p+1);
        n1 = atoi(arg1);n2 = atoi(arg2);
    }
    /* Make the response body */
    sprintf(content, "Welcome to add.com: ");
    sprintf(content, "%sTHE Internet addition portal.\r\n<p>", content);
    sprintf(content, "%sThe answer is: %d + %d = %d\r\n<p>",
        content, n1, n2, n1 + n2);
    sprintf(content, "%sThanks for visiting!\r\n", content);
    /* Generate the HTTP response */
    printf("Connection: close\r\n");
    printf("Content-length: %d\r\n", (int)strlen(content));
    //注意此处空行
    printf("Content-type: text/html\r\n\r\n");
    printf("%s", content);
    fflush(stdout);
```

Tiny 服务器

```
Rio_readlineb(&rio, buf, MAXLINE)
sscanf(buf, "%s %s %s", method, uri, version);

if (stat(filename, &sbuf) < 0) {                                //line:netp:doit:beginnotfound
    clienterror(fd, filename, "404", "Not found",
               "Tiny couldn't find this file");
    return;
}                                                                //line:netp:doit:endnotfound

if (is_static) { /* Serve static content */
    if (!(S_ISREG(sbuf.st_mode)) || !(S_IRUSR & sbuf.st_mode)) { //line:netp:doit:readable
        clienterror(fd, filename, "403", "Forbidden",
                     "Tiny couldn't read the file");
        return;
    }
    serve_static(fd, filename, sbuf.st_size);                  //line:netp:doit:servestatic
}

else { /* Serve dynamic content */
    if (!(S_ISREG(sbuf.st_mode)) || !(S_IXUSR & sbuf.st_mode)) { //line:netp:doit:executable
        clienterror(fd, filename, "403", "Forbidden",
                     "Tiny couldn't run the CGI program");
        return;
    }
}
```

Tiny 服务器

```
int parse_uri(char *uri, char *filename, char *cgiargs)
{
    char *ptr;

    if (!strstr(uri, "cgi-bin")) { /* Static content */ //line:netp:parseuri:isstatic
        strcpy(cgiargs, ""); //line:netp:parseuri:clearcgi
        strcpy(filename, "."); //line:netp:parseuri:beginconvert1
        strcat(filename, uri); //line:netp:parseuri:endconvert1
        if (uri[strlen(uri)-1] == '/') //line:netp:parseuri:slashcheck
            strcat(filename, "home.html"); //line:netp:parseuri:appenddefault
        return 1;
    }
    else { /* Dynamic content */ //line:netp:parseuri:isdynamic
        ptr = index(uri, '?'); //line:netp:parseuri:beginextract
        if (ptr) {
            strcpy(cgiargs, ptr+1);
            *ptr = '\0';
        }
        else
            strcpy(cgiargs, ""); //line:netp:parseuri:endextract
        strcpy(filename, "."); //line:netp:parseuri:beginconvert2
        strcat(filename, uri); //line:netp:parseuri:endconvert2
        return 0;
    }
}
```

Tiny 服务器

```
void serve_static(int fd, char *filename, int filesize)
{
    int srcfd;
    char *srcp, filetype[MAXLINE], buf[MAXBUF];

    /* Send response headers to client */
    get_filetype(filename, filetype);    //line:netp:serve_static:getfiletype
    sprintf(buf, "HTTP/1.0 200 OK\r\n"); //line:netp:serve_static:beginserve
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Server: Tiny Web Server\r\n");
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Content-length: %d\r\n", filesize);
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Content-type: %s\r\n\r\n", filetype);
    Rio_writen(fd, buf, strlen(buf));    //line:netp:serve_static:endserve

    /* Send response body to client */
    srcfd = Open(filename, O_RDONLY, 0); //line:netp:serve_static:open
    srcp = Mmap(0, filesize, PROT_READ, MAP_PRIVATE, srcfd, 0); //line:netp:serve_static:mmap
    Close(srcfd);                //line:netp:serve_static:close
    Rio_writen(fd, srcp, filesize); //line:netp:serve_static:write
    Munmap(srcp, filesize);        //line:netp:serve_static:munmap
}
```

Tiny 服务器

```
void serve_dynamic(int fd, char *filename, char *cgiargs)
{
    char buf[MAXLINE], *emptylist[] = { NULL };

    /* Return first part of HTTP response */
    sprintf(buf, "HTTP/1.0 200 OK\r\n");
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf, "Server: Tiny Web Server\r\n");
    Rio_writen(fd, buf, strlen(buf));

    if (Fork() == 0) { /* Child */ //line:netp:servedynamic:fork
        /* Real server would set all CGI vars here */
        setenv("QUERY_STRING", cgiargs, 1); //line:netp:servedynamic:setenv
        Dup2(fd, STDOUT_FILENO);           /* Redirect stdout to client */ //line:netp:servedynamic:dup2
        Execve(filename, emptylist, environ); /* Run CGI program */ //line:netp:servedynamic:execve
    }
    Wait(NULL); /* Parent waits for and reaps child */ //line:netp:servedynamic:wait
}
```

Homework Review

Exercises

E1

11. 下列关于全球 IP 因特网的描述中**正确**的是:

- A. IP 协议提供了一种从进程到进程的递送机制。
- B. TCP 协议提供了进程间可靠的全双工连接。
- C. IP 地址结构中地址的字节顺序是大端法还是小端法, 要根据主机字节顺序来确定。
- D. 多个域名不可能被映射为同一个 IP 地址。

E1

11. 下列关于全球 IP 因特网的描述中**正确**的是:

- A. IP 协议提供了一种从进程到进程的递送机制。
- B. TCP 协议提供了进程间可靠的全双工连接。
- C. IP 地址结构中地址的字节顺序是大端法还是小端法, 要根据主机字节顺序来确定。
- D. 多个域名不可能被映射为同一个 IP 地址。

E2

12. 套接字接口结合 Unix I/O 函数，可以创建网络应用。下列相关描述中，错误的是：

- A. `socket` 函数成功返回描述符以后，还不能直接用于读写。
- B. TCP 连接可以由连接双方的套接字对四元组（双方的 IP 地址和端口号）唯一确定。
- C. `bind` 函数用于将服务端的主动套接字转化为监听套接字。
- D. `connect` 函数会阻塞，直到连接成功建立或者发生错误。

E2

答案： C

A. `socket()` 函数创建一个新的套接字，并返回一个文件描述符，这个套接字只是被创建出来，并不是立即可用于读写数据。要进行读写操作，需要额外的步骤，如调用 `bind()` 绑定地址、`listen()` 启动监听（对于服务器端），或者 `connect()` 建立连接（对于客户端）。

B. TCP 连接是通过四元组唯一标识的，这个四元组包括：客户端的 IP 地址、客户端的端口号、服务器的 IP 地址、服务器的端口号。这四个信息组合在一起能唯一确定一个 TCP 连接。

C. 错误，应为 `listen` 函数。`bind()` 函数只是绑定套接字到特定的地址和端口，套接字是否监听则是通过 `listen()` 来实现的。

D. `connect()` 函数用于建立与服务器的连接。对于客户端而言，`connect()` 是一个阻塞调用，直到连接成功建立或者出现错误（例如服务器不可达）时才会返回。

E3

17. 以下符合对 TCP 协议描述的是

①可靠传输

②在主机间传输

③可以基于 IP 协议

④全双工

⑤可以基于 DNS

A. ①②④

B. ③④⑤

C. ①③④

D. ②④

E3

17. 以下符合对 TCP 协议描述的是

- ①可靠传输 ②在主机间传输 ③可以基于 IP 协议
- ④全双工 ⑤可以基于 DNS

- A. ①②④ B. ③④⑤ C. ①③④ D. ②④

答案：C←

难度：简单←

预计时间：小于 1min←

①TCP 提供可靠传输←

②TCP 可以在两个进程间传输，包括同一个主机上的←

③TCP/IP 本身是一个协议族，TCP 可以基于 IP 来实现，也存在不使用 IP 的实现←

④TCP 是全双工的，双方同时都可以进行读写←

⑤TCP 并不依赖 DNS 服务←

E4

18. 对于建立客户端与服务器的 TCP 连接, 以下关于套接字接口, 正确的是:
- A. 对于 TCP 连接而言, 其可以由连接双方的套接字对四元组 (双方的 IP 地址和端口号) 唯一确定。
 - B. `connect` 函数用于客户端建立与服务器的连接, 其参数中的套接字地址结构需要指明本客户端使用的端口号
 - C. `socket` 函数返回的套接字描述符, 可以立即使用标准 Unix I/O 函数进行读写
 - D. `bind` 函数用于将用于客户端的主动套接字转化为用于服务器的监听套接字

E4

18. 对于建立客户端与服务器的 TCP 连接, 以下关于套接字接口, 正确的是:
- A. 对于 TCP 连接而言, 其可以由连接双方的套接字对四元组 (双方的 IP 地址和端口号) 唯一确定。
 - B. connect 函数用于客户端建立与服务器的连接, 其参数中的套接字地址结构需要指明本客户端使用的端口号
 - C. socket 函数返回的套接字描述符, 可以立即使用标准 Unix I/O 函数进行读写
 - D. bind 函数用于将用于客户端的主动套接字转化为用于服务器的监听套接字

答案: A↵

难度: 简单↵

预计时间: 小于 1min↵

A. TCP 连接由连接双方进程的套接字唯一确定↵

B. connect 应该指明想要连接的服务器的套接字地址结构↵

C. socket 只是获得套接字, 还不可以进行读写, 需要完成连接以后才可以进行读写↵

D. bind 函数只是用于绑定套接字地址的, 而 listen 函数用于将用于客户端的主动套接字转化为用于服务器的监听套接字↵

E5

4. 下列关于全球 IP 因特网的说法中，正确的是↵
- A. TCP/IP 协议规定网络字节顺序取决于不同主机的字节顺序，从而方便主机和网络之间收发数据↵
 - B. 域名集合和 IP 地址集合的映射关系目前由 http 来维护↵
 - C. 多个域名可以映射到同一个 IP 地址，但一个域名不可以映射到多个 IP 地址↵
 - D. 从内核的角度来看，套接字是通信的一个端点；从用户程序的角度来看，套接字是一个有相应描述符的打开文件↵

E5

4. 下列关于全球 IP 因特网的说法中，正确的是↵
- A. TCP/IP 协议规定网络字节顺序取决于不同主机的字节顺序，从而方便主机和网络之间收发数据↵
 - B. 域名集合和 IP 地址集合的映射关系目前由 http 来维护↵
 - C. 多个域名可以映射到同一个 IP 地址，但一个域名不可以映射到多个 IP 地址↵
 - D. 从内核的角度来看，套接字是通信的一个端点；从用户程序的角度来看，套接字是一个有相应描述符的打开文件↵

答案：D↵

解析：↵

A：网络字节顺序是大端法↵

B：DNS 负责维护域名集合和 IP 地址集合的映射关系

C：一个域名也可以被映射到多个 IP 地址↵

E6

5. 下列关于网络的说法中，错误的是↵
- A. 在 TCP 连接中，客户端和服务端都是指主机↵
 - B. 网络按照覆盖范围的大小可以分成局域网和广域网，其中一种流行的局域网技术是以太网↵
 - C. 网络可以被主机认为是一种 I/O 设备，是数据源和数据接收方↵
 - D. 路由器可以把多个不兼容的网络连接起来组成一个互联网络↵

E6

5. 下列关于网络的说法中，错误的是↵
- A. 在 TCP 连接中，客户端和服务端都是指主机↵
 - B. 网络按照覆盖范围的大小可以分成局域网和广域网，其中一种流行的局域网技术是以太网↵
 - C. 网络可以被主机认为是一种 I/O 设备，是数据源和数据接收方↵
 - D. 路由器可以把多个不兼容的网络连接起来组成一个互联网络↵

答案：A↵

解析：简单题，考察 11.1 部分内容↵

A：在该模型中，客户端和服务端指进程↵

BCD 均正确↵

E7

20. 按照课程中的描述，套接字 socket 接口是一组函数，它们与 UNIX I/O 函数一起使用可以构建网络应用。下列关于 socket 的描述中，哪一个是错误的？
- A. 因特网的套接字地址存放在 `socketaddr_in` 结构中
 - B. 在使用套接字接口函数时，要将对应协议的特定结构指针转换成通用结构 `sockaddr`
 - C. 当客户端通过 `connect()` 发起连接请求时，由服务器端的服务进程通过 `listen()` 捕获该请求并建立监听套接字 `listenfd`
 - D. 对于迭代 echo 服务器，当 `client1` 连接到该服务器且尚未断开连接时，`client2` 若想要连接该服务器会阻塞在从服务器读结果的操作上

E7

20. 按照课程中的描述，套接字 socket 接口是一组函数，它们与 UNIX I/O 函数一起使用可以构建网络应用。下列关于 socket 的描述中，哪一个是错误的？

- A. 因特网的套接字地址存放在 `socketaddr_in` 结构中
- B. 在使用套接字接口函数时，要将对应协议的特定结构指针转换成通用结构 `sockaddr`
- C. 当客户端通过 `connect()` 发起连接请求时，由服务器端的服务进程通过 `listen()` 捕获该请求并建立监听套接字 `listenfd`
- D. 对于迭代 echo 服务器，当 `client1` 连接到该服务器且尚未断开连接时，`client2` 若想要连接该服务器会阻塞在从服务器读结果的操作上

20. C。服务器端的进程通过 `accept` 来建立已连接套接字。D 应阻塞在 `read`，参看课程投影片。

E8

21. 下列关于全球 IP 因特网的描述中，哪一个是不正确的？
- A. IPV4 协议中，一个 IP 地址是一个 32 位无符号整数
 - B. IP 地址结构中存放的地址总是以大端法存放的
 - C. 网络字节顺序是采用大端法的
 - D. 在 Linux 系统上，用 `hostname -i` 命令可以确定自己主机的域名

E8

21. 下列关于全球 IP 因特网的描述中，哪一个是不正确的？

- A. IPV4 协议中，一个 IP 地址是一个 32 位无符号整数
- B. IP 地址结构中存放的地址总是以大端法存放的
- C. 网络字节顺序是采用大端法的
- D. 在 Linux 系统上，用 `hostname -i` 命令可以确定自己主机的域名

21. D。送分题，根据这个命令的字面意思也能看出获得的是主机名，参数 `-i` 表示返回点分十进制的 IP 地址。网络字节顺序和大端法是一回事。

E9

14. 采用迭代的 Echo 服务器服务多个客户端，假设 `client1` 先完成连接，并且没有结束，那么 `client2` 向服务器发出连接请求后，会被阻塞在下列哪个函数上？
- A. `connect`
 - B. `read`
 - C. `write`
 - D. `socket`

E9

14. 采用迭代的 Echo 服务器服务多个客户端，假设 `client1` 先完成连接，并且没有结束，那么 `client2` 向服务器发出连接请求后，会被阻塞在下列哪个函数上？
- A. `connect`
 - B. `read`
 - C. `write`
 - D. `socket`

14. B。参看课程投影片，迭代服务器的第二个客户端会在 `read` 阻塞。

E10

15. 下列关于全球 IP 因特网的各种概念叙述中，哪一个是错误的？
- A. 一个套接字是连接的一个端点，对应了一个套接字地址
 - B. IPv4 的因特网中，一个 IP 地址是一个 32 位无符号整数
 - C. 因特网主机的字节顺序与 TCP/IP 定义的网络字节顺序是一致的
 - D. IP 协议提供了一种不可靠的从一台主机向其他主机的递送机制

E10

15. 下列关于全球 IP 因特网的各种概念叙述中，哪一个是错误的？

- A. 一个套接字是连接的一个端点，对应了一个套接字地址
- B. IPv4 的因特网中，一个 IP 地址是一个 32 位无符号整数
- C. 因特网主机的字节顺序与 TCP/IP 定义的网络字节顺序是一致的
- D. IP 协议提供了一种不可靠的从一台主机向其他主机的递送机制

15. C。网络字节序规定为大端序，在不同的主机中可能需要转换。

E11

10. 下面有关计算机网络概念的叙述中，**正确**的是
- A. 大写字母的 Internet 用来描述互联网的一般概念，而小写字母的 internet 用来描述一种具体的实现，也就是全球 IP 互联网。
 - B. 在一个基于集线器(hub)的以太网(Ethernet)中，如果往一台主机发送一段数据帧(frame)，那么其他主机无法看到这个帧。
 - C. IP 协议提供基本的命名方法和递送机制，因此我们能够借助 IP 协议，从一台主机往另一台主机发送包，即使两台主机不在同一个 LAN 内。
 - D. 当一段数据通过路由器，从 LAN1 被发送到 LAN2 时，附加的互联网络包头和局域网帧头保持不变。

E11

答案：C

A 选项出自书本 645 页，考察互联网的基本概念。internet 应该是描述一般概念，Internet 应该是特指一个具体实现

B 选项出自书本 643-644 页，考察 Ethernet 协议的概念和实现形式。早期的 Ethernet 协议使用 Hub 来连接网络主机，所有的帧传送都是广播形式。

C 选项出自书本 647 页，考察 IP 协议在网络协议栈中的角色。

D 选项出自书本 645-646 页，考察网络协议的基本概念。在通过路由器将数据从一个局域网传送到另一个局域网的过程中，原来的局域网帧头会被剥离，然后附加新的局域网帧头

E12

11. 下面有关套接字接口(Socket API)的叙述中, **错误**的是
- A. 套接字接口常常被用来创建网络应用
 - B. Windows 10 系统没有实现套接字接口
 - C. `getaddrinfo()` 和 `getnameinfo()` 可以被用于编写独立于特定版本的 IP 协议的程序
 - D. `socket()` 函数返回的描述符, 可以使用标准 Unix I/O 函数进行读写

E12

11. 下面有关套接字接口(Socket API)的叙述中, **错误**的是
- A. 套接字接口常常被用来创建网络应用
 - B. Windows 10 系统没有实现套接字接口
 - C. `getaddrinfo()` 和 `getnameinfo()` 可以被用于编写独立于特定版本的 IP 协议的程序
 - D. `socket()` 函数返回的描述符, 可以使用标准 Unix I/O 函数进行读写

答案: B

A 选项是套接字的基本概念

B 选项出自书本 652 页, 考察对套接字接口的理解。几乎所有的现代操作系统都实现了套接字接口。

C 选项出自书本 656 页, 考察套接字接口中协议无关的设计思想。

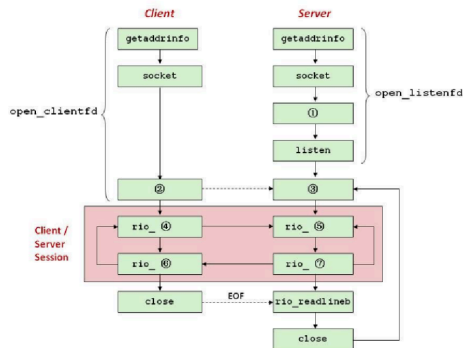
D 选项考察对 Socket API 和 Linux 文件描述符的基本理解

E13

第六题 (10 分)

下图是一个基于 echo 服务器的 client-server 框架

(1) 请给图中的编号填写相应的函数名。



(2) 请补全下面 server 端 open_listendf 函数中缺失的操作 (Line 21, Line 26 和 Line 34)

```

Line 1: int open_listendf(char *port)
Line 2: {
Line 3:     struct addrinfo hints, *listp, *p;
Line 4:     int listendfd, optval=1;
Line 5:     /* Get a list of potential server addresses */
Line 6:     memset(&hints, 0, sizeof(struct addrinfo));
Line 7:     hints.ai_socktype = SOCK_STREAM;
Line 8:     hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG;
Line 9:     hints.ai_flags |= AI_NUMERICSERV;
Line 10:    Getaddrinfo(NULL, port, &hints, &listp);

```

```

Line 11:
Line 12:    for (p = listp; p; p = p->ai_next) {
Line 13:        /* Create a socket descriptor */
Line 14:        if ((listendfd = socket(p->ai_family, p->ai_socktype,
Line 15:                                p->ai_protocol)) < 0)
Line 16:            continue; /* Socket failed, try the next */
Line 17:        /* Eliminates "Address already in use" error from
Line 18:        bind */
Line 19:        Setsockopt(listendfd, SOL_SOCKET, SO_REUSEADDR,
Line 20:                    (const void *)&optval , sizeof(int));
Line 21:
Line 22:        if ( ( (21) (listendfd, p->ai_addr, p->ai_addrlen) == 0)
Line 23:            break; /* Success */
Line 24:        Close(listendfd);
Line 25:    }
Line 26:    /* Clean up */
Line 27:    Freeaddrinfo( (26) );
Line 28:    if (!p) /* No address worked */
Line 29:        return -1;
Line 30:
Line 31:    if (listen(listendfd, LISTENQ) < 0) {
Line 32:        Close(listendfd);
Line 33:        return -1;
Line 34:    }
Line 35: }

```

E13

解答题五

1. `bind`, `connect`, `accept`, `writen`, `readlineb`, `readlineb`, `writen`。容易读解，但注意 RIO 包的 API 名称不能记错。
2. `bind`, `listp`, `listenfd`。

E14

第六题 (10 分)

1. 请在以下的表格中填入相应内容，每个空格仅需填一项。其中第 1 列可填选项包括：网络层、传输层、应用层；第 2 列可填选项 IP、UDP、TCP、HTTP；第 3 列可填选项包括：是、否。

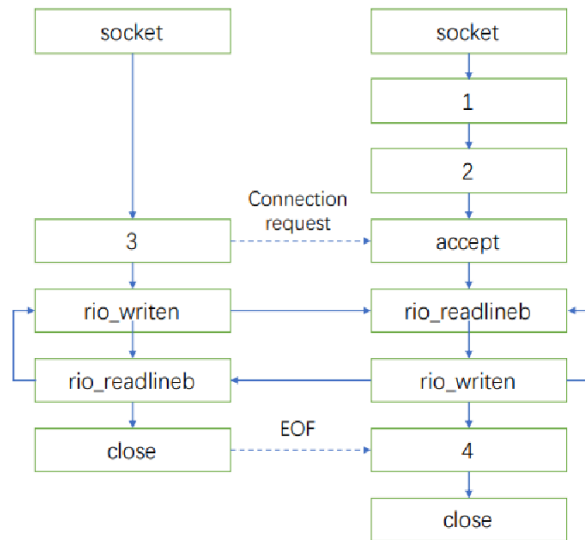
(说明：面向连接的协议保障数据按照发送时的顺序被接收)

协议层次	协议名称	是面向连接的吗?
网络层		
	HTTP	
	TCP	

答案：(每空 1 分，共 6 分)

协议层次	协议名称	是面向连接的吗?
网络层	IP	否
应用层	HTTP	否
传输层	TCP	是

2. 下图描述了客户端与服务器套接字连接和通信的过程，请在空格处补充相关步骤。



答案：(每空 1 分，共 4 分)

1: bind

2: listen

3: connect

4: rio_readlineb

Notices

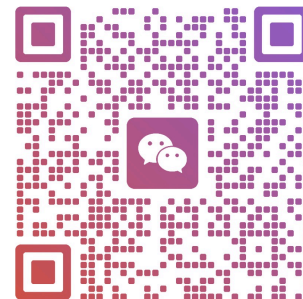
Malloclab Style

THANKS

Made by WEB-05

webrun@stu.pku.edu.cn

Reference: [WalkerCH]'s and [Arthals]'s presentations.



扫一扫上面的二维码图案，加我为朋友。